Listing 5.19 `tmdb.py` (`get_top_movies_by_genre`)

```
@kernel_function(     #1
        description="""
Gets a list of currently playing movies for a given genre""",
        name="get_top_movies_by_genre",
        input_description="The genre of the movies to get",
        )
    def get_top_movies_by_genre(self, genre: str) -> str:
        print_function_call()
        genre_id = self.get_movie_genre_id(genre)     #2
        if genre_id:
            base_url = "https://api.themoviedb.org/3
            playing_movies_endpoint = f"{base_url}/movie/now_playing?↪
↪ api_key={self.api_key}&language=en-US"
            response = requests.get(
                        playing_movies_endpoint)     #3
            if response.status_code != 200:
                return ""

            playing_movies = response.json()['results'
            for movie in playing_movies:     #4
                movie['genre_ids'] = [str(genre_id)
                      ↪ for genre_id in movie['genre_ids']]
            filtered_movies = [movie for movie ↪
↪ in playing_movies if genre_id ↪
↪ in movie['genre_ids']][:10]     #5
            results = ", ".join([movie['title'] for movie in filtered_movies])
            return results
        else:
            return ""
```

#1 Decorates the function with descriptions
#2 Finds the genre id for the given genre name
#3 Gets a list of currently playing movies
#4 Converts genre_ids to strings
#5 Checks to see if the genre id matches movie genres

Look through the various other API calls mapped semantically. As you can see, there is a well-defined pattern for converting API calls to a semantic service. Before we run the full service, we'll test each of the functions in the next section.

5.5.2 Testing semantic services

In a real-world application, you'll likely want to write a complete set of unit or integration tests for each semantic service function. We won't do that here; instead, we'll write a quick helper script to test the various functions.